RELATÓRIO DE TRABALHO PRÁTICO

Implementação de sistema cadastral com interface Web, baseado nas tecnologias de Servlets, JPA e JEE

1 INTRODUÇÃO

Este relatório descreve o desenvolvimento do trabalho prático da disciplina DGT2811 – Desenvolvimento Back-End Corporativo com Java e Cloud. A proposta consiste na construção de um sistema cadastral de produtos para a plataforma Java Web, organizado em camadas e apoiado nas especificações JPA, EJB, Servlets e JSP, com o framework Bootstrap aplicado à interface.

O trabalho foi dividido em três procedimentos. No primeiro foram definidas as camadas de persistência e de controle, a partir do banco de dados loja criado em prática anterior. No segundo foi construída a interface cadastral, com um Servlet no padrão Front Controller e duas páginas JSP. No terceiro, as páginas receberam as classes do Bootstrap para melhoria do design. Ao fim de cada procedimento são apresentados os códigos, os resultados da execução e a análise solicitada no roteiro.

O código-fonte completo, incluindo o script do banco de dados e este relatório, está disponível no repositório https://github.com/souzafgabriel/CadastroEE.

2 OBJETIVOS

2.1 Objetivo geral

Desenvolver um aplicativo corporativo Web na plataforma Jakarta EE capaz de consultar, incluir, alterar e excluir produtos armazenados em um banco SQL Server.

2.2 Objetivos específicos

- implementar a persistência de dados com base em JPA;
- implementar as regras de negócio por meio de EJBs do tipo Session Bean;
- construir a interface cadastral com Servlets e JSPs;
- utilizar a biblioteca Bootstrap para melhorar o design das páginas.

3 MATERIAIS E MÉTODOS

3.1 Ambiente de desenvolvimento

O desenvolvimento e os testes foram realizados em ambiente Windows, com as ferramentas relacionadas a seguir.

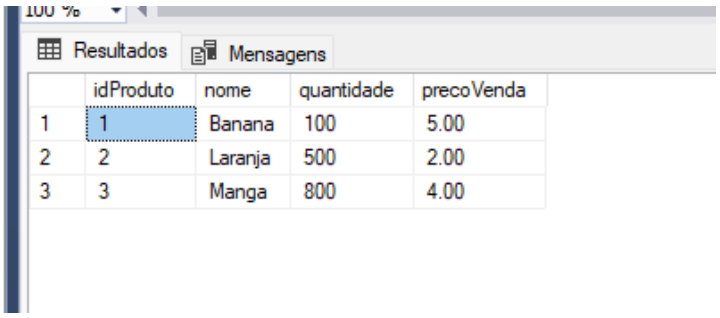
- **IDE:** Apache NetBeans 25, com projeto do tipo Java with Ant › Java Enterprise › Enterprise Application;
- **servidor de aplicação:** GlassFish Server 6.2.5 (Jakarta EE 9.1), executado sobre o JDK 17 (Eclipse Temurin);
- **banco de dados:** Microsoft SQL Server 2022 Express, administrado pelo SQL Server Management Studio 20;
- **driver JDBC:** mssql-jdbc-12.2.0.jre8;
- **front-end:** JSP com Bootstrap 5.3.3 carregado via CDN.

Duas adaptações em relação ao roteiro precisaram ser feitas. A primeira diz respeito à versão da plataforma: com o GlassFish 6.2.5, o NetBeans 25 oferece apenas a opção Jakarta EE 9.1, que já utiliza o namespace jakarta.*. Por esse motivo, a inclusão das bibliotecas Jakarta EE 8 API e a substituição manual dos imports javax por jakarta, previstas nos itens 4 e 6 do primeiro procedimento, não foram necessárias. A segunda refere-se à porta HTTP do servidor, alterada de 8080 para 8081 porque a porta padrão já estava ocupada por outro serviço na máquina. Por isso os endereços de teste deste relatório utilizam a porta 8081.

3.2 Banco de dados

Foi utilizado o banco loja, com as tabelas Usuario, Pessoa, PessoaFisica, PessoaJuridica, Produto e Movimento. O acesso é feito pelo login loja, com senha loja, em autenticação do SQL Server. O script de criação encontra-se em sql/loja.sql no repositório. A Figura 1 mostra os produtos cadastrados inicialmente.

Figura 1 – Produtos cadastrados no banco loja



	idProduto	nome	quantidade	precoVenda
1	1	Banana	100	5.00
2	2	Laranja	500	2.00
3	3	Manga	800	4.00

Fonte: elaborado pelo autor (2026).

3.3 Estrutura do projeto

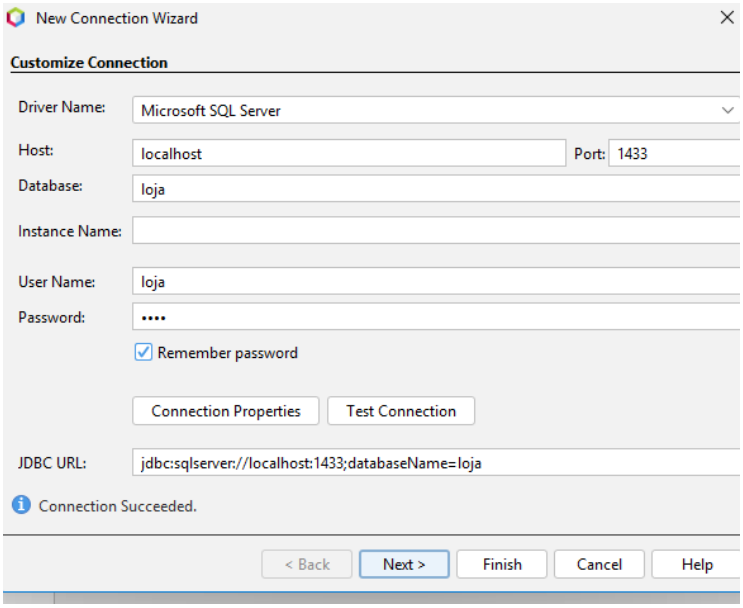
O projeto corporativo CadastroEE gera um arquivo EAR e é composto por dois módulos. O módulo CadastroEE-ejb reúne as entidades JPA, no pacote cadastroee.model, e os Session Beans, no pacote cadastroee.controller. O módulo CadastroEE-war contém os Servlets, no pacote cadastroee.servlets, e as páginas JSP.

4 1º PROCEDIMENTO – CAMADAS DE PERSISTÊNCIA E CONTROLE

4.1 Conexão com o banco e pool de conexões

O driver JDBC do SQL Server foi registrado no NetBeans, na aba Services › Databases › Drivers, e em seguida foi criada a conexão com o banco loja. Como o SQL Server utiliza certificado autoassinado, foi preciso incluir a opção trustServerCertificate=true na URL de conexão. O teste da conexão é apresentado na Figura 2.

Figura 2 – Teste da conexão com o banco loja no NetBeans



Fonte: elaborado pelo autor (2026).

O mesmo arquivo do driver foi copiado para o diretório lib do GlassFish. Com o servidor em execução, o pool de conexões e o recurso JNDI foram criados pelo utilitário asadmin, conforme o Código 1. O comando de criação do pool foi dividido em duas etapas, create e set, porque o PowerShell não interpreta corretamente os caracteres de escape da propriedade URL. As propriedades driverClass e URL não são usadas pelo GlassFish, mas são lidas pelo assistente Entity Classes from Database do NetBeans para acessar o banco a partir do data source.

Código 1 – Configuração do pool de conexões e do recurso JNDI no asadmin

```
asadmin create-jdbc-connection-pool
  --datasourceclassname com.microsoft.sqlserver.jdbc.SQLServerDataSource
  --restype javax.sql.DataSource
  --property
serverName=localhost:portNumber=1433:databaseName=loja:user=loja:password=loja:trustServerCert
ificate=true
  SQLServerPool
JDBC connection pool SQLServerPool created successfully.
```

```
asadmin set resources.jdbc-connection-
pool.SQLServerPool.property.driverClass=com.microsoft.sqlserver.jdbc.SQLServerDriver
asadmin set "resources.jdbc-connection-pool.SQLServerPool.property.URL=jdbc:sqlserver://
localhost:1433;databaseName=loja;encrypt=true;trustServerCertificate=true;"
Command set executed successfully.

asadmin ping-connection-pool SQLServerPool
Command ping-connection-pool executed successfully.

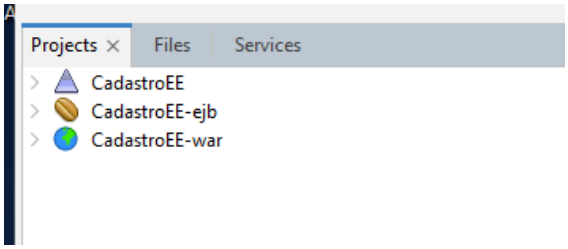
asadmin create-jdbc-resource --connectionpoolid SQLServerPool jdbc/loja
JDBC resource jdbc/loja created successfully.
```

Fonte: elaborado pelo autor (2026).

4.2 Criação do projeto e camada de persistência

O projeto CadastroEE foi criado como Enterprise Application, com o GlassFish como servidor e os módulos CadastroEE-ejb e CadastroEE-war (Figura 3). No módulo EJB, as entidades foram geradas pelo assistente New Entity Classes from Database, a partir do data source jdbc/loja, selecionando as seis tabelas do banco. O mesmo assistente criou o arquivo persistence.xml.

Figura 3 – Projetos gerados pelo assistente Enterprise Application



Fonte: elaborado pelo autor (2026).

Na entidade Produto foram feitos dois ajustes manuais. O atributo precoVenda passou de BigDecimal para Float, conforme o roteiro, e o atributo idProduto recebeu a anotação @GeneratedValue(strategy = GenerationType.IDENTITY). Sem essa anotação, o JPA envia um identificador nulo na inclusão, o que provoca erro, já que a coluna é do tipo IDENTITY no SQL Server. O persistence.xml foi substituído pelo modelo indicado no roteiro (Código 2) e a entidade Produto final é apresentada no Código 3.

Código 2 – Arquivo persistence.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<persistence version="1.0"
             xmlns="http://java.sun.com/xml/ns/persistence"
             xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
             xsi:schemaLocation="http://java.sun.com/xml/ns/persistence
http://java.sun.com/xml/ns/persistence/persistence_1_0.xsd">
  <persistence-unit name="CadastroEE-ejbPU" transaction-type="JTA">
    <jta-data-source>jdbc/loja</jta-data-source>
    <exclude-unlisted-classes>false</exclude-unlisted-classes>
    <properties/>
```

```
  </persistence-unit>
</persistence>
```

Fonte: elaborado pelo autor (2026).

Código 3 – Entidade Produto

```
package cadastroee.model;

import java.io.Serializable;
import java.util.Collection;
import jakarta.persistence.Basic;
import jakarta.persistence.Column;
import jakarta.persistence.Entity;
import jakarta.persistence.GeneratedValue;
import jakarta.persistence.GenerationType;
import jakarta.persistence.Id;
import jakarta.persistence.NamedQueries;
import jakarta.persistence.NamedQuery;
import jakarta.persistence.OneToMany;
import jakarta.persistence.Table;

@Entity
@Table(name = "Produto")
@NamedQueries({
    @NamedQuery(name = "Produto.findAll", query = "SELECT p FROM Produto p ORDER BY
p.idProduto"),
    @NamedQuery(name = "Produto.findByIdProduto", query = "SELECT p FROM Produto p WHERE
p.idProduto = :idProduto"),
    @NamedQuery(name = "Produto.findByNome", query = "SELECT p FROM Produto p WHERE p.nome
= :nome"),
    @NamedQuery(name = "Produto.findByQuantidade", query = "SELECT p FROM Produto p WHERE
p.quantidade = :quantidade"),
    @NamedQuery(name = "Produto.findByPrecoVenda", query = "SELECT p FROM Produto p WHERE
p.precoVenda = :precoVenda")})
public class Produto implements Serializable {

    private static final long serialVersionUID = 1L;

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    @Basic(optional = false)
    @Column(name = "idProduto")
    private Integer idProduto;

    @Basic(optional = false)
    @Column(name = "nome")
    private String nome;

    @Basic(optional = false)
    @Column(name = "quantidade")
    private int quantidade;

    @Basic(optional = false)
    @Column(name = "precoVenda")
    private Float precoVenda;

    @OneToMany(mappedBy = "idProduto")
    private Collection<Movimento> movimentoCollection;
```

```
public Produto() {
}

public Produto(Integer idProduto) {
    this.idProduto = idProduto;
}

public Produto(Integer idProduto, String nome, int quantidade, Float precoVenda) {
    this.idProduto = idProduto;
    this.nome = nome;
    this.quantidade = quantidade;
    this.precoVenda = precoVenda;
}

public Integer getIdProduto() {
    return idProduto;
}

public void setIdProduto(Integer idProduto) {
    this.idProduto = idProduto;
}

public String getNome() {
    return nome;
}

public void setNome(String nome) {
    this.nome = nome;
}

public int getQuantidade() {
    return quantidade;
}

public void setQuantidade(int quantidade) {
    this.quantidade = quantidade;
}

public Float getPrecoVenda() {
    return precoVenda;
}

public void setPrecoVenda(Float precoVenda) {
    this.precoVenda = precoVenda;
}

public Collection<Movimento> getMovimentoCollection() {
    return movimentoCollection;
}

public void setMovimentoCollection(Collection<Movimento> movimentoCollection) {
    this.movimentoCollection = movimentoCollection;
}

@Override
public int hashCode() {
    int hash = 0;
```

```
        hash += (idProduto != null ? idProduto.hashCode() : 0);
        return hash;
    }

    @Override
    public boolean equals(Object object) {
        if (!(object instanceof Produto)) {
            return false;
        }
        Produto other = (Produto) object;
        return !((this.idProduto == null && other.idProduto != null)
                || (this.idProduto != null && !this.idProduto.equals(other.idProduto)));
    }

    @Override
    public String toString() {
        return "cadastroee.model.Produto[ idProduto=" + idProduto + " ]";
    }
}
```

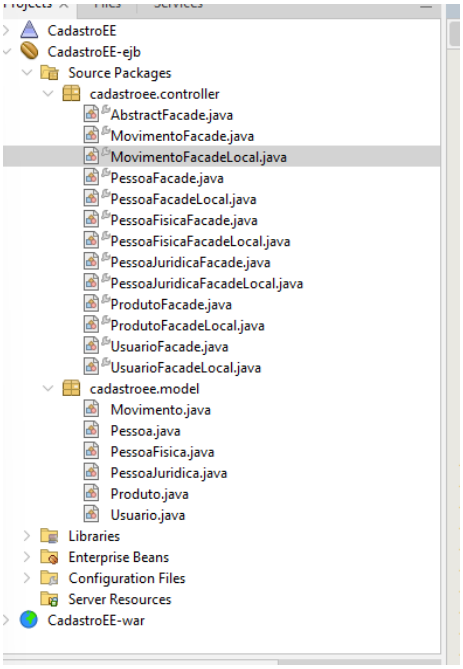
Fonte: elaborado pelo autor (2026).

As demais entidades (Movimento, Pessoa, PessoaFisica, PessoaJuridica e Usuario) foram mantidas como geradas pelo NetBeans e podem ser consultadas no repositório.

4.3 Camada de controle

Os componentes EJB foram gerados pelo assistente New Session Beans for Entity Classes, com interface local e pacote cadastroee.controller. O assistente criou a classe abstrata AbstractFacade, que concentra as operações de CRUD, e um par Facade/FacadeLocal para cada entidade (Figura 4). Os Códigos 4 a 6 apresentam as classes relacionadas a Produto.

Figura 4 – Entidades e Session Beans gerados no módulo CadastroEE-ejb



Fonte: elaborado pelo autor (2026).

Código 4 – Classe AbstractFacade

```
package cadastroee.controller;

import java.util.List;
import jakarta.persistence.EntityManager;
import jakarta.persistence.criteria.CriteriaQuery;
import jakarta.persistence.criteria.Root;

public abstract class AbstractFacade<T> {

    private final Class<T> entityClass;

    public AbstractFacade(Class<T> entityClass) {
        this.entityClass = entityClass;
    }

    protected abstract EntityManager getEntityManager();

    public void create(T entity) {
        getEntityManager().persist(entity);
    }

    public void edit(T entity) {
        getEntityManager().merge(entity);
    }

    public void remove(T entity) {
        getEntityManager().remove(getEntityManager().merge(entity));
    }

    public T find(Object id) {
        return getEntityManager().find(entityClass, id);
    }
```

```
    public List<T> findAll() {
        CriteriaQuery<T> cq =
getEntityManager().getCriteriaBuilder().createQuery(entityClass);
        cq.select(cq.from(entityClass));
        return getEntityManager().createQuery(cq).getResultList();
    }

    public List<T> findRange(int[] range) {
        CriteriaQuery<T> cq =
getEntityManager().getCriteriaBuilder().createQuery(entityClass);
        cq.select(cq.from(entityClass));
        jakarta.persistence.TypedQuery<T> q = getEntityManager().createQuery(cq);
        q.setMaxResults(range[1] - range[0] + 1);
        q.setFirstResult(range[0]);
        return q.getResultList();
    }

    public int count() {
        CriteriaQuery<Long> cq =
getEntityManager().getCriteriaBuilder().createQuery(Long.class);
        Root<T> rt = cq.from(entityClass);
        cq.select(getEntityManager().getCriteriaBuilder().count(rt));
        return getEntityManager().createQuery(cq).getSingleResult().intValue();
    }
}
```

Fonte: elaborado pelo autor (2026).

Código 5 – Interface ProdutoFacadeLocal

```
package cadastroee.controller;

import cadastroee.model.Produto;
import java.util.List;
import jakarta.ejb.Local;

@Local
public interface ProdutoFacadeLocal {

    void create(Produto produto);

    void edit(Produto produto);

    void remove(Produto produto);

    Produto find(Object id);

    List<Produto> findAll();

    List<Produto> findRange(int[] range);

    int count();
}
```

Fonte: elaborado pelo autor (2026).

Código 6 – Session Bean ProdutoFacade

```
package cadastroee.controller;
```

```
import cadastroee.model.Produto;
import jakarta.ejb.Stateless;
import jakarta.persistence.EntityManager;
import jakarta.persistence.PersistenceContext;

@Stateless
public class ProdutoFacade extends AbstractFacade<Produto> implements ProdutoFacadeLocal {

    @PersistenceContext(unitName = "CadastroEE-ejbPU")
    private EntityManager em;

    @Override
    protected EntityManager getEntityManager() {
        return em;
    }

    public ProdutoFacade() {
        super(Produto.class);
    }
}
```

Fonte: elaborado pelo autor (2026).

4.4 Servlet de teste

No módulo CadastroEE-war foi criado o ServletProduto, no pacote cadastroee.servlets. O Session Bean é injetado pela anotação @EJB e o método processRequest recupera os produtos e os apresenta em uma lista HTML (Código 7). Para que o WAR enxergasse as classes do módulo EJB, o projeto CadastroEE-ejb foi adicionado às bibliotecas de compilação do CadastroEE-war, sem empacotamento, já que o JAR integra o EAR.

O arquivo web.xml seguiu o modelo do roteiro, acrescido dos elementos servlet-mapping, sem os quais os endereços dos Servlets retornam erro 404 (Código 8).

Código 7 – Servlet ServletProduto

```
package cadastroee.servlets;

import cadastroee.controller.ProdutoFacadeLocal;
import cadastroee.model.Produto;
import java.io.IOException;
import java.io.PrintWriter;
import java.util.List;
import jakarta.ejb.EJB;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

public class ServletProduto extends HttpServlet {

    @EJB
    ProdutoFacadeLocal facade;
```

```
    protected void processRequest(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        response.setContentType("text/html;charset=UTF-8");
        try (PrintWriter out = response.getWriter()) {
            out.println("<!DOCTYPE html>");
            out.println("<html>");
            out.println("<head>");
            out.println("<meta charset=\"UTF-8\">");
            out.println("<title>Servlet ServletProduto</title>");
            out.println("</head>");
            out.println("<body>");
            out.println("<h1>Servlet ServletProduto at " + request.getContextPath() +
"</h1>");
            out.println("<h2>Produtos</h2>");
            out.println("<ul>");
            List<Produto> produtos = facade.findAll();
            for (Produto p : produtos) {
                out.println("<li>" + p.getNome() + "</li>");
            }
            out.println("</ul>");
            out.println("</body>");
            out.println("</html>");
        }
    }

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        processRequest(request, response);
    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        processRequest(request, response);
    }

    @Override
    public String getServletInfo() {
        return "Short description";
    }
}
```

Fonte: elaborado pelo autor (2026).

Código 8 – Descritor web.xml

```
<?xml version="1.0" encoding="UTF-8"?>
<web-app version="4.0" xmlns="http://xmlns.jcp.org/xml/ns/javaee"
         xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
         xsi:schemaLocation="http://xmlns.jcp.org/xml/ns/javaee
http://xmlns.jcp.org/xml/ns/javaee/web-app_4_0.xsd">
    <servlet>
        <servlet-name>ServletProduto</servlet-name>
        <servlet-class>cadastroee.servlets.ServletProduto</servlet-class>
    </servlet>
    <servlet>
        <servlet-name>ServletProdutoFC</servlet-name>
```

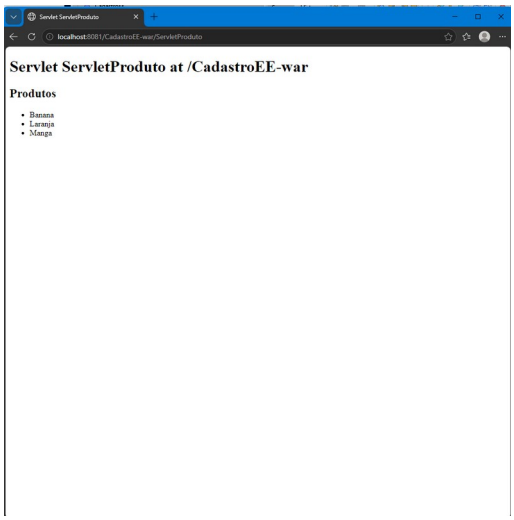
```
        <servlet-class>cadastroee.servlets.ServletProdutoFC</servlet-class>
    </servlet>
    <servlet-mapping>
        <servlet-name>ServletProduto</servlet-name>
        <url-pattern>/ServletProduto</url-pattern>
    </servlet-mapping>
    <servlet-mapping>
        <servlet-name>ServletProdutoFC</servlet-name>
        <url-pattern>/ServletProdutoFC</url-pattern>
    </servlet-mapping>
    <session-config>
        <session-timeout>30</session-timeout>
    </session-config>
    <welcome-file-list>
        <welcome-file>index.html</welcome-file>
    </welcome-file-list>
</web-app>
```

Fonte: elaborado pelo autor (2026).

4.5 Resultados

O projeto foi executado pela opção Run do projeto principal. O acesso ao endereço http://localhost:8081/CadastroEE-war/ServletProduto exibiu os produtos cadastrados no banco, o que confirma o funcionamento conjunto do pool de conexões, da camada JPA e do Session Bean (Figura 5).

Figura 5 – Saída do ServletProduto



Fonte: elaborado pelo autor (2026).

4.6 Análise e conclusão

4.6.1 Organização de um projeto corporativo no NetBeans

O projeto do tipo Enterprise Application é formado por um projeto principal e por módulos dependentes. O projeto principal, aqui chamado CadastroEE, não contém código próprio: sua função é empacotar o arquivo EAR. O módulo EJB gera um arquivo JAR com as entidades, o persistence.xml e os Session Beans, enquanto o

módulo Web gera o WAR com Servlets, páginas JSP e o web.xml. O NetBeans controla a dependência entre os módulos e os scripts Ant de compilação, de modo que a execução do projeto principal compila, empacota e implanta toda a aplicação no servidor configurado.

4.6.2 Papel das tecnologias JPA e EJB

A JPA responde pela persistência. Por meio de anotações como @Entity, @Table, @Column e @Id, as classes Java são mapeadas para as tabelas do banco, e o EntityManager executa inclusões, alterações, exclusões e consultas sem que o desenvolvedor precise escrever SQL (DEVMEDIA, 2023b). O EJB, por sua vez, abriga as regras de negócio. Os Session Beans são gerenciados pelo container, que oferece pool de instâncias, injeção do EntityManager, controle transacional via JTA e segurança (GEEKSFORGEEKS, 2023). Com isso, a camada Web consome serviços prontos e não precisa lidar com conexões ou transações.

4.6.3 Produtividade do NetBeans com JPA e EJB

O ganho de produtividade vem principalmente dos assistentes de geração de código. O New Entity Classes from Database lê os metadados das tabelas e cria as entidades com atributos, relacionamentos, consultas nomeadas e o persistence.xml. O New Session Beans for Entity Classes gera o AbstractFacade, os Facades e suas interfaces. No caso deste trabalho, as seis entidades e as treze classes de controle foram obtidas em poucos passos. A integração com o GlassFish, que permite iniciar o servidor, implantar a aplicação e consultar os recursos JDBC dentro da própria IDE, também reduz o tempo de desenvolvimento.

4.6.4 Servlets e o suporte do NetBeans

Servlets são classes Java que estendem HttpServlet e são executadas pelo container Web. Elas recebem requisições HTTP pelos métodos doGet e doPost e produzem respostas dinâmicas. O NetBeans disponibiliza um assistente que cria a classe já com o método processRequest e com doGet e doPost delegando a ele, além de registrar o Servlet e seu mapeamento no web.xml quando a opção Add information to deployment descriptor é marcada.

4.6.5 Comunicação entre Servlets e Session Beans

A comunicação ocorre por injeção de dependência. O Servlet declara um atributo do tipo da interface local anotado com @EJB, e o container injeta uma referência ao bean ao inicializar o componente. Cada chamada de método, como facade.findAll(), é atendida por uma instância livre do pool de Stateless Session Beans, dentro de uma transação gerenciada pelo container. Como os dois módulos

fazem parte do mesmo EAR, a chamada é local, ocorre na mesma JVM e dispensa serialização.

5 2º PROCEDIMENTO – INTERFACE CADASTRAL COM SERVLET E JSPS

5.1 Servlet ServletProdutoFC

O ServletProdutoFC foi implementado segundo o padrão Front Controller. O parâmetro acao define a operação a executar (listar, incluir, alterar, excluir, formIncluir ou formAlterar) e também a página de destino: ProdutoDados.jsp para os formulários e ProdutoLista.jsp para as demais ações. Ao final, a requisição é encaminhada com RequestDispatcher.forward. Foi acrescentado o tratamento de entradas inválidas e de falhas na exclusão, como a tentativa de excluir um produto referenciado na tabela Movimento. Nesses casos o usuário retorna à listagem com uma mensagem de erro, em vez de receber uma página de erro do servidor (Código 9).

Código 9 – Servlet ServletProdutoFC

```
package cadastroee.servlets;

import cadastroee.controller.ProdutoFacadeLocal;
import cadastroee.model.Produto;
import java.io.IOException;
import jakarta.ejb.EJB;
import jakarta.servlet.RequestDispatcher;
import jakarta.servlet.ServletException;
import jakarta.servlet.http.HttpServlet;
import jakarta.servlet.http.HttpServletRequest;
import jakarta.servlet.http.HttpServletResponse;

public class ServletProdutoFC extends HttpServlet {

    @EJB
    ProdutoFacadeLocal facade;

    protected void processRequest(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        request.setCharacterEncoding("UTF-8");

        String acao = request.getParameter("acao");
        if (acao == null || acao.isBlank()) {
            acao = "listar";
        }

        String destino = ("formAlterar".equals(acao) || "formIncluir".equals(acao))
                ? "ProdutoDados.jsp"
                : "ProdutoLista.jsp";

        try {
            switch (acao) {
                case "listar":
                    break;

                case "formIncluir":
                    break;

                case "formAlterar": {
```

```
                    Integer id = Integer.valueOf(request.getParameter("id"));
                    request.setAttribute("produto", facade.find(id));
                    break;
                }

                case "excluir": {
                    Integer id = Integer.valueOf(request.getParameter("id"));
                    Produto produto = facade.find(id);
                    if (produto != null) {
                        facade.remove(produto);
                        request.setAttribute("mensagem", "Produto excluído com sucesso.");
                    }
                    break;
                }

                case "alterar": {
                    Integer id = Integer.valueOf(request.getParameter("id"));
                    Produto produto = facade.find(id);
                    if (produto != null) {
                        preencher(produto, request);
                        facade.edit(produto);
                        request.setAttribute("mensagem", "Produto alterado com sucesso.");
                    }
                    break;
                }

                case "incluir": {
                    Produto produto = new Produto();
                    preencher(produto, request);
                    facade.create(produto);
                    request.setAttribute("mensagem", "Produto incluído com sucesso.");
                    break;
                }

                default:
                    request.setAttribute("erro", "Ação inválida: " + acao);
                    destino = "ProdutoLista.jsp";
            }
        } catch (NumberFormatException e) {
            request.setAttribute("erro", "Dados inválidos: verifique quantidade, preço e
id.");
            destino = "ProdutoLista.jsp";
        } catch (Exception e) {
            request.setAttribute("erro", "Não foi possível concluir a operação \"" + acao
                    + "\". O produto pode estar vinculado a movimentações.");
            destino = "ProdutoLista.jsp";
            log("Erro em ServletProdutoFC, acao=" + acao, e);
        }

        if ("ProdutoLista.jsp".equals(destino)) {
            request.setAttribute("produtos", facade.findAll());
        }

        RequestDispatcher rd = request.getRequestDispatcher(destino);
        rd.forward(request, response);
    }

    private void preencher(Produto produto, HttpServletRequest request) {
```

```
        produto.setNome(request.getParameter("nome").trim());
        produto.setQuantidade(Integer.parseInt(request.getParameter("quantidade").trim()));

produto.setPrecoVenda(Float.valueOf(request.getParameter("precoVenda").trim().replace(",",
"."")));
    }

    @Override
    protected void doGet(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        processRequest(request, response);
    }

    @Override
    protected void doPost(HttpServletRequest request, HttpServletResponse response)
            throws ServletException, IOException {
        processRequest(request, response);
    }

    @Override
    public String getServletInfo() {
        return "Short description";
    }
}
```

Fonte: elaborado pelo autor (2026).

5.2 Página ProdutoLista.jsp

A página de consulta recupera a lista de produtos enviada pelo Servlet e apresenta, em uma tabela, o identificador, o nome, a quantidade e o preço de cada item. Para cada linha são gerados os links de alteração e exclusão, com a ação e o id correspondentes. O Código 10 mostra a versão do segundo procedimento, ainda sem Bootstrap.

Código 10 – Página ProdutoLista.jsp (versão sem Bootstrap)

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@page import="java.util.List"%>
<%@page import="java.util.Locale"%>
<%@page import="cadastroee.model.Produto"%>
<!DOCTYPE html>
<html lang="pt-br">
    <head>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1">
        <title>Lista de Produtos</title>
    </head>
    <body>
        <h1>Listagem de Produtos</h1>

        <%
            String mensagem = (String) request.getAttribute("mensagem");
            String erro = (String) request.getAttribute("erro");
            if (mensagem != null) {
        %>
```

```
        <div role="alert"><%= mensagem %></div>
        <%  }
            if (erro != null) { %>
        <div role="alert"><%= erro %></div>
        <%  } %>

        <a href="ServletProdutoFC?acao=formIncluir">Novo Produto</a>

        <table border="1">
            <thead>
                <tr>
                    <th>#</th>
                    <th>Nome</th>
                    <th>Quantidade</th>
                    <th>Preço de Venda</th>
                    <th>Opções</th>
                </tr>
            </thead>
            <tbody>
                <%
                    List<Produto> produtos = (List<Produto>) request.getAttribute("produtos");
                    if (produtos != null) {
                        for (Produto p : produtos) {
                %>
                <tr>
                    <td><%= p.getIdProduto() %></td>
                    <td><%= p.getNome() %></td>
                    <td><%= p.getQuantidade() %></td>
                    <td><%= String.format(new Locale("pt", "BR"), "R$ %.2f",
p.getPrecoVenda()) %></td>
                    <td>
                        <a
                           href="ServletProdutoFC?acao=formAlterar&id=<%= p.getIdProduto()
%>">Alterar</a>
                        <a
                           href="ServletProdutoFC?acao=excluir&id=<%= p.getIdProduto() %>"
                           onclick="return confirm('Excluir o produto <%=
p.getNome().replace("'", "\\'") %>?');">Excluir</a>
                    </td>
                </tr>
                <%
                        }
                    }
                %>
            </tbody>
        </table>
    </body>
</html>
```

Fonte: elaborado pelo autor (2026).

5.3 Página ProdutoDados.jsp

A página de cadastro contém um formulário enviado por POST ao ServletProdutoFC. Quando nenhuma entidade é recebida, a variável acao assume o valor incluir; quando um produto é fornecido, assume o valor alterar e os campos são

preenchidos com os dados atuais. O id é enviado em campo oculto apenas no modo de alteração, e o texto do botão acompanha a operação (Código 11).

Código 11 – Página ProdutoDados.jsp (versão sem Bootstrap)

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@page import="cadastroee.model.Produto"%>
<!DOCTYPE html>
<html lang="pt-br">
    <head>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1">
        <title>Dados do Produto</title>
    </head>
    <body>
        <%
            Produto produto = (Produto) request.getAttribute("produto");
            String acao = (produto == null) ? "incluir" : "alterar";
        %>
        <h1>Dados do Produto</h1>

        <form action="ServletProdutoFC" method="post">
            <input type="hidden" name="acao" value="<%= acao %>">
            <% if ("alterar".equals(acao)) { %>
            <input type="hidden" name="id" value="<%= produto.getIdProduto() %>">
            <% } %>

            <div>
                <label for="nome">Nome</label>
                <input type="text" id="nome" name="nome" required maxlength="255"
                       value="<%= (produto != null) ? produto.getNome().replace("\"",
"&quot;") : "" %>">
            </div>

            <div>
                <label for="quantidade">Quantidade</label>
                <input type="number" id="quantidade" name="quantidade" required min="0"
step="1"
                       value="<%= (produto != null) ? produto.getQuantidade() : "" %>">
            </div>

            <div>
                <label for="precoVenda">Preço de Venda</label>
                <input type="number" id="precoVenda" name="precoVenda" required min="0"
step="0.01"
                       value="<%= (produto != null) ? produto.getPrecoVenda() : "" %>">
            </div>

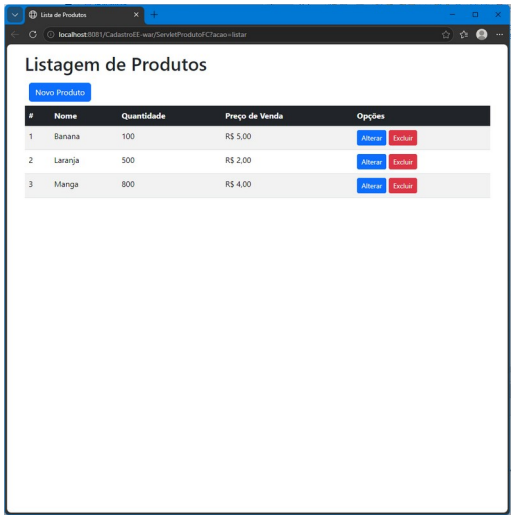
            <button type="submit"><%= "incluir".equals(acao) ? "Incluir" : "Alterar"
%></button>
            <a href="ServletProdutoFC?acao=listar">Voltar</a>
        </form>
    </body>
</html>
```

Fonte: elaborado pelo autor (2026).

5.4 Resultados

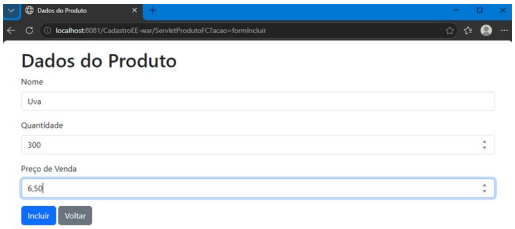
Os testes foram realizados com a versão final das páginas, já com Bootstrap, uma vez que o fluxo do Front Controller é o mesmo nas duas versões. Partindo da listagem inicial (Figura 6), foi incluído o produto Uva, com 300 unidades a R$ 6,50 (Figuras 7 e 8). Em seguida sua quantidade foi alterada para 350 (Figuras 9 e 10) e, por fim, o produto foi excluído (Figura 11). Em todas as operações a listagem refletiu corretamente o estado do banco.

Figura 6 – Listagem inicial de produtos (acao=listar)



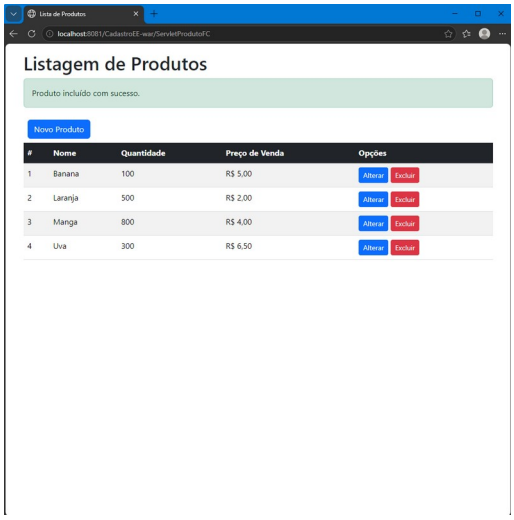
Fonte: elaborado pelo autor (2026).

Figura 7 – Formulário de inclusão preenchido



Fonte: elaborado pelo autor (2026).

Figura 8 – Listagem após a inclusão



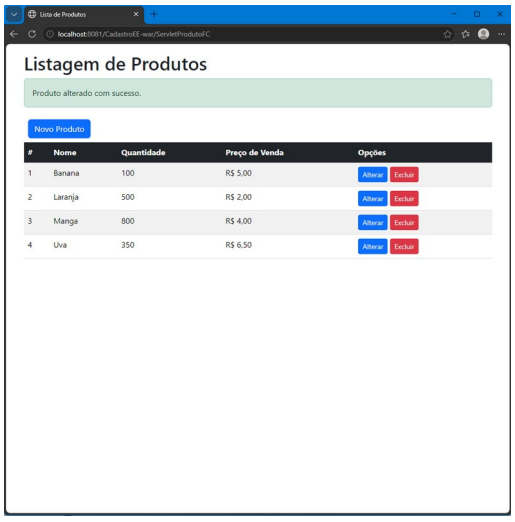
Fonte: elaborado pelo autor (2026).

Figura 9 – Formulário de alteração do produto 4



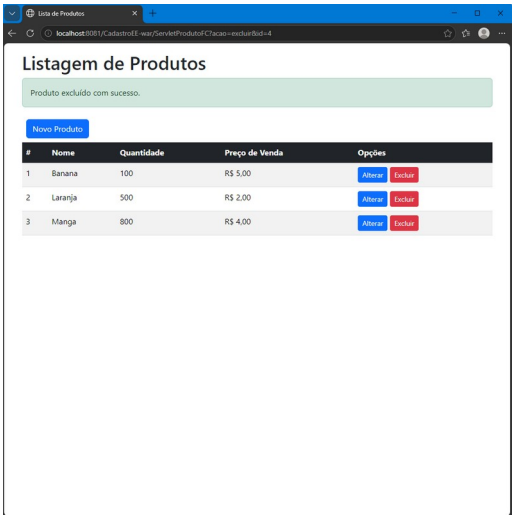
Fonte: elaborado pelo autor (2026).

Figura 10 – Listagem após a alteração



Fonte: elaborado pelo autor (2026).

Figura 11 – Listagem após a exclusão



Fonte: elaborado pelo autor (2026).

5.5 Análise e conclusão

5.5.1 O padrão Front Controller na arquitetura MVC

O Front Controller estabelece um ponto único de entrada para as requisições de um módulo. Em vez de existir um Servlet para cada operação, um único controlador recebe as chamadas, identifica a ação desejada, aciona o modelo e escolhe a visão que exibirá o resultado. Na arquitetura MVC aplicada neste trabalho, o controlador é o ServletProdutoFC, o modelo é formado pelas entidades JPA e pelo ProdutoFacade, e a visão corresponde às páginas JSP, que apenas apresentam os atributos recebidos. A centralização facilita o tratamento de erros e a navegação e evita que a visão acesse diretamente a camada de dados.

5.5.2 Diferenças e semelhanças entre Servlets e JSPs

Servlets e JSPs são componentes Web executados no servidor e têm acesso aos mesmos objetos implícitos, como request, response e session. Na prática, todo JSP é traduzido e compilado em um Servlet na primeira requisição. A diferença está na forma de escrita. No Servlet, o HTML é produzido por código Java, o que o torna adequado à lógica de controle. No JSP, o código Java aparece embutido em um documento HTML, o que favorece a apresentação. Por essa razão, no MVC, os Servlets costumam atuar como controladores e os JSPs como visões.

5.5.3 Redirecionamento simples, forward e o uso de parâmetros e atributos

No redirecionamento simples, feito com response.sendRedirect, o servidor responde com o código HTTP 302 e o navegador realiza uma nova requisição para outro endereço. A URL exibida muda e os dados da requisição original se perdem. No forward, realizado pelo RequestDispatcher, o encaminhamento ocorre dentro do

servidor: o mesmo request é repassado ao recurso de destino, a URL permanece a mesma e os atributos são preservados. É por isso que o ServletProdutoFC utiliza o forward para entregar a lista de produtos ao JSP.

Parâmetros e atributos têm funções distintas. Os parâmetros são valores do tipo String enviados pelo cliente, seja na URL ou nos campos de um formulário, e são lidos com getParameter. Os atributos são objetos de qualquer tipo definidos no servidor com setAttribute e recuperados com getAttribute. Eles servem para transportar dados entre componentes durante o processamento da requisição, como ocorre com a lista de produtos repassada do Servlet para a página de listagem.

6 3º PROCEDIMENTO – MELHORIA DO DESIGN DA INTERFACE

6.1 Inclusão do Bootstrap

Os links do Bootstrap 5.3.3 obtidos no site oficial, referentes à folha de estilos e ao pacote JavaScript, foram incluídos na seção head das duas páginas. Em ProdutoLista.jsp, o body recebeu a classe container, o link de inclusão as classes btn, btn-primary e m-2, a tabela as classes table e table-striped, o cabeçalho a classe table-dark, e os links de alteração e exclusão as classes btn-primary e btn-danger com o tamanho btn-sm. Em ProdutoDados.jsp, cada par de rótulo e campo foi colocado em uma div com a classe mb-3, o formulário recebeu a classe form, os rótulos a classe form-label, os campos a classe form-control e o botão as classes btn e btn-primary. Também foram adicionados alertas para as mensagens de sucesso e de erro enviadas pelo Servlet (Códigos 12 e 13).

Código 12 – Página ProdutoLista.jsp (versão final)

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@page import="java.util.List"%>
<%@page import="java.util.Locale"%>
<%@page import="cadastroee.model.Produto"%>
<!DOCTYPE html>
<html lang="pt-br">
    <head>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1">
        <title>Lista de Produtos</title>
        <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet" crossorigin="anonymous">
        <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"
crossorigin="anonymous"></script>
    </head>
    <body class="container">
        <h1 class="mt-3">Listagem de Produtos</h1>

        <%
            String mensagem = (String) request.getAttribute("mensagem");
            String erro = (String) request.getAttribute("erro");
            if (mensagem != null) {
        %>
        <div class="alert alert-success" role="alert"><%= mensagem %></div>
        <%  }
            if (erro != null) { %>
        <div class="alert alert-danger" role="alert"><%= erro %></div>
        <%  } %>

        <a class="btn btn-primary m-2" href="ServletProdutoFC?acao=formIncluir">Novo
Produto</a>

        <table class="table table-striped">
            <thead class="table-dark">
                <tr>
```

```
                    <th>#</th>
                    <th>Nome</th>
                    <th>Quantidade</th>
                    <th>Preço de Venda</th>
                    <th>Opções</th>
                </tr>
            </thead>
            <tbody>
                <%
                    List<Produto> produtos = (List<Produto>) request.getAttribute("produtos");
                    if (produtos != null) {
                        for (Produto p : produtos) {
                %>
                <tr>
                    <td><%= p.getIdProduto() %></td>
                    <td><%= p.getNome() %></td>
                    <td><%= p.getQuantidade() %></td>
                    <td><%= String.format(new Locale("pt", "BR"), "R$ %.2f",
p.getPrecoVenda()) %></td>
                    <td>
                        <a class="btn btn-primary btn-sm"
                           href="ServletProdutoFC?acao=formAlterar&id=<%= p.getIdProduto()
%>">Alterar</a>
                        <a class="btn btn-danger btn-sm"
                           href="ServletProdutoFC?acao=excluir&id=<%= p.getIdProduto() %>"
                           onclick="return confirm('Excluir o produto <%=
p.getNome().replace("'", "\\'") %>?');">Excluir</a>
                    </td>
                </tr>
                <%
                        }
                    }
                %>
            </tbody>
        </table>
    </body>
</html>
```

Fonte: elaborado pelo autor (2026).

Código 13 – Página ProdutoDados.jsp (versão final)

```
<%@page contentType="text/html" pageEncoding="UTF-8"%>
<%@page import="cadastroee.model.Produto"%>
<!DOCTYPE html>
<html lang="pt-br">
    <head>
        <meta charset="UTF-8">
        <meta name="viewport" content="width=device-width, initial-scale=1">
        <title>Dados do Produto</title>
        <link href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet" crossorigin="anonymous">
        <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min.js"
crossorigin="anonymous"></script>
    </head>
    <body class="container">
        <%
            Produto produto = (Produto) request.getAttribute("produto");
```

```
            String acao = (produto == null) ? "incluir" : "alterar";
        %>
        <h1 class="mt-3">Dados do Produto</h1>

        <form class="form" action="ServletProdutoFC" method="post">
            <input type="hidden" name="acao" value="<%= acao %>">
            <% if ("alterar".equals(acao)) { %>
            <input type="hidden" name="id" value="<%= produto.getIdProduto() %>">
            <% } %>

            <div class="mb-3">
                <label for="nome" class="form-label">Nome</label>
                <input type="text" class="form-control" id="nome" name="nome" required
maxlength="255"
                       value="<%= (produto != null) ? produto.getNome().replace("\"",
"&quot;") : "" %>">
            </div>

            <div class="mb-3">
                <label for="quantidade" class="form-label">Quantidade</label>
                <input type="number" class="form-control" id="quantidade" name="quantidade"
required min="0" step="1"
                       value="<%= (produto != null) ? produto.getQuantidade() : "" %>">
            </div>

            <div class="mb-3">
                <label for="precoVenda" class="form-label">Preço de Venda</label>
                <input type="number" class="form-control" id="precoVenda" name="precoVenda"
required min="0" step="0.01"
                       value="<%= (produto != null) ? produto.getPrecoVenda() : "" %>">
            </div>

            <button type="submit" class="btn btn-primary"><%= "incluir".equals(acao) ?
"Incluir" : "Alterar" %></button>
            <a class="btn btn-secondary" href="ServletProdutoFC?acao=listar">Voltar</a>
        </form>
    </body>
</html>
```

Fonte: elaborado pelo autor (2026).

6.2 Resultados

O resultado visual pode ser observado nas Figuras 6 a 11, apresentadas no procedimento anterior. A Figura 8, em especial, reúne os principais elementos aplicados à listagem: container centralizado, botão de inclusão, tabela listrada com cabeçalho escuro, botões de ação e alerta de sucesso. Nos formulários (Figuras 7 e 9), os campos ocupam a largura do container e mantêm espaçamento uniforme entre si.

6.3 Análise e conclusão

6.3.1 Utilização do Bootstrap

O Bootstrap é incorporado à página pela inclusão de sua folha de estilos e de seu arquivo JavaScript, que neste trabalho foram carregados de uma CDN. A partir daí, o visual é obtido com a aplicação de classes predefinidas aos elementos HTML, sem necessidade de escrever CSS próprio. O JavaScript habilita os componentes interativos do framework, como modais e menus suspensos (BOOTSTRAP, 2023).

6.3.2 Independência estrutural do HTML

Como toda a aparência é definida por classes CSS, a marcação HTML permanece responsável apenas pela estrutura e pelo significado do conteúdo. O visual é aplicado de forma externa e pode ser modificado com a troca de classes ou de tema, sem alterar a estrutura das páginas nem a lógica do Servlet. Isso ficou evidente neste trabalho: as mesmas páginas do segundo procedimento passaram a ter outro design apenas com a adição de classes.

6.3.3 Bootstrap e responsividade

O Bootstrap adota a abordagem mobile-first. Seus estilos são definidos primeiro para telas pequenas e ajustados para telas maiores por meio de media queries, organizadas em pontos de quebra (sm, md, lg, xl e xxl). O sistema de grid, os containers e as classes utilitárias responsivas fazem o layout se reorganizar de acordo com a largura do dispositivo. Em conjunto com a meta tag viewport, presente nas duas páginas, isso permite que o cadastro seja utilizado em computadores, tablets e celulares sem código específico para cada um.

7 CONSIDERAÇÕES FINAIS

O trabalho resultou em um aplicativo corporativo organizado em camadas. A persistência foi implementada com JPA sobre o SQL Server, por meio de um pool de conexões registrado no GlassFish. As regras de negócio ficaram a cargo de Session Beans com transações gerenciadas pelo container, o controle foi centralizado em um Servlet no padrão Front Controller e a visão foi construída em páginas JSP estilizadas com Bootstrap. Todas as operações de consulta, inclusão, alteração e exclusão foram testadas com sucesso.

Durante a execução surgiram algumas dificuldades de ambiente que não constam no roteiro, como a configuração da autenticação e da porta do SQL Server, a escolha da versão do JDK compatível com o GlassFish 6.2.5 e a necessidade das propriedades driverClass e URL no pool para que o assistente do NetBeans pudesse ler as tabelas. A resolução desses pontos contribuiu para a compreensão de como os componentes da plataforma se integram.

Como possíveis melhorias, destacam-se a aplicação do padrão Post/Redirect/Get após inclusões e alterações, para evitar o reenvio do formulário ao atualizar a página, a realização da exclusão por requisição POST e a substituição dos scriptlets por JSTL e Expression Language nas páginas JSP.

REFERÊNCIAS

BOOTSTRAP. *Bootstrap docs: introduction*. [S. l.], 2023. Disponível em: https://getbootstrap.com/docs/5.0/getting-started/introduction/. Acesso em: 24 out. 2026.

DEVMEDIA. *JDBC tutorial*. [S. l.], 2023a. Disponível em: https://www.devmedia.com.br/jdbc-tutorial/6638. Acesso em: 24 out. 2026.

DEVMEDIA. *Introdução à JPA – Java Persistence API*. [S. l.], 2023b. Disponível em: https://www.devmedia.com.br/introducao-a-jpa-java-persistence-api/28173. Acesso em: 24 out. 2026.

ESTÁCIO. *Programação servidor com Java*. Rio de Janeiro: Estácio, [s. d.]. Disponível em: https://stensineme.blob.core.windows.net/hmlgrepoh/00212ti/00905/index.html. Acesso em: 24 out. 2026.

ESTÁCIO. *Tecnologias JPA e JEE*. Rio de Janeiro: Estácio, [s. d.]. Disponível em: https://stensineme.blob.core.windows.net/hmlgrepoh/00212ti/00965/index.html. Acesso em: 24 out. 2026.

ESTÁCIO. *Trabalho prático DGT2811: desenvolvimento back-end corporativo com Java e Cloud*. Rio de Janeiro: Estácio, 2025. Roteiro da disciplina.

GEEKSFORGEEKS. *Enterprise Java Beans (EJB)*. [S. l.], 2023. Disponível em: https://www.geeksforgeeks.org/enterprise-java-beans-ejb/. Acesso em: 24 out. 2026.